

Università degli Studi di Perugia



Dipartimento di Matematica e Informatica

Relazione / Documentazione Tecnica – Cinemanova

Studente

Dedi Vittorio (Mat. 357 339)

Anno Accademico 2024-2025

Panoramica

L'idea che ha guidato questo progetto è stata quella di creare un sito web in grado di gestire in modo completo tutti i servizi offerti da un cinema multisala. L'obiettivo principale era sviluppare una piattaforma che fosse utile sia agli utenti finali, cioè i clienti, sia al personale interno, facilitando così sia la fruizione dei servizi che la gestione operativa.

Per realizzare questa applicazione ho scelto di utilizzare Blazor Server, più avanti entrerà nel dettaglio delle motivazioni tecniche di questa scelta, così come delle strategie adottate per l'implementazione. Per quanto riguarda l'archiviazione dei dati, invece, ho optato per SQLite, un database leggero e molto semplice da integrare, che si adatta perfettamente alle esigenze di un sistema di questo tipo.

Il sistema è strutturato in modo da offrire tre livelli di accesso, ciascuno con funzionalità specifiche e pensate su misura per i diversi tipi di utenti:

1. Utente non registrato: chiunque può visitare il sito senza bisogno di autenticarsi e ha accesso a informazioni di base come il catalogo dei film in programmazione, gli orari delle proiezioni, le sale disponibili e le informazioni generali sul cinema.
2. Utente autenticato (cliente): chi invece si registra e accede con il proprio account ha, oltre a tutto ciò che può fare un utente non registrato, la possibilità di verificare in tempo reale la disponibilità dei posti per ogni proiezione, completare l'acquisto dei biglietti e scaricare direttamente il biglietto in formato PDF. Questo biglietto digitale potrà poi essere mostrato all'ingresso della sala. La generazione e il download del PDF sono gestiti tramite una funzione JavaScript che permette di salvare il file sul dispositivo in modo semplice e immediato.
3. Utente amministratore: Può gestire in modo completo la programmazione delle proiezioni, modificando o cancellando eventi, aggiornare il catalogo film sfruttando anche l'integrazione con l'API di The Movie Database (TMDb) per facilitare l'inserimento dei dati, e amministrare gli account degli utenti. L'accesso alle pagine riservate agli amministratori è protetto tramite sistemi di autorizzazione specifici, garantendo la sicurezza dell'intero sistema.

Scelte Tecniche

Per quanto riguarda la tecnologia, la decisione di adottare Blazor Server è stata presa perché mi permette di mantenere sempre sincronizzata la disponibilità dei posti in sala: ogni prenotazione o rilascio viene propagato istantaneamente a tutti i client connessi tramite SignalR. Ho trovato inoltre estremamente pratico poter ospitare front-end, API, autenticazione

e logiche di business (inclusa l'integrazione con TMDb) all'interno di un'unica applicazione. Sul fronte della sicurezza, non dover mai esporre le chiavi API nel bundle scaricato dal browser ha tolto ogni necessità di realizzare proxy o back-end dedicati solo allo scopo di mascherare segreti: tutte le chiamate a TMDb vengono gestite sul server e le credenziali restano confinate negli user-secrets.

Per quanto concerne la scelta del database, SQLite si è rivelata la soluzione più adatta per la sua leggerezza e facilità di configurazione, senza necessità di un server esterno. Questo garantisce buone prestazioni in locale per le operazioni tipiche di un sistema di biglietteria, con un numero di transazioni simultanee adeguato al contesto, oltre a una grande portabilità dei dati per backup o migrazioni.

L'integrazione con l'API di TMDb è stata particolarmente utile per gestire in modo automatico e preciso i dati dei film: grazie a questa API è possibile recuperare in modo semplice e veloce informazioni dettagliate come locandine, sinossi, cast e trailer, facilitando molto il processo di inserimento manuale e migliorando la qualità complessiva del catalogo.

Infine, ho scelto di utilizzare componenti di MudBlazor per migliorare l'esperienza utente con elementi di interfaccia più moderni e funzionali, come le stelle per le valutazioni. Per quanto riguarda l'interfaccia e lo styling, ho adottato un approccio modulare, rimuovendo il file NavMenu.razor e centralizzando la navigazione in MainLayout.razor. Ogni pagina ha un file CSS dedicato, così da mantenere separati e ordinati gli stili. Gran parte delle icone sono prese da Bootstrap Icons, integrate direttamente nei componenti Blazor.

Sul fronte della sicurezza, l'autenticazione si basa su cookie gestiti da ASP.NET Core, con sessioni che durano massimo 30 minuti e si aggiornano automaticamente se l'utente resta attivo. La validazione dei dati inseriti avviene sia lato client sia lato server, utilizzando le classi ViewModel con annotazioni di validazione standard.

Struttura del Database

Il database, realizzato con SQLite, si basa su alcune tabelle principali che riflettono le entità chiave dell'applicazione:

- **UserAccounts:** qui sono memorizzati gli account degli utenti, con campi come l'ID, email, nome utente, password e ruolo (User o Amministratore).
- **Films:** contiene tutte le informazioni sui film disponibili, dai titoli alle date di uscita, generi, registi, durate, sinossi, locandine e un flag che indica se la proiezione è attiva o meno e voto.
- **Sale:** definisce le varie sale del cinema, con nome, numero identificativo e capienza.

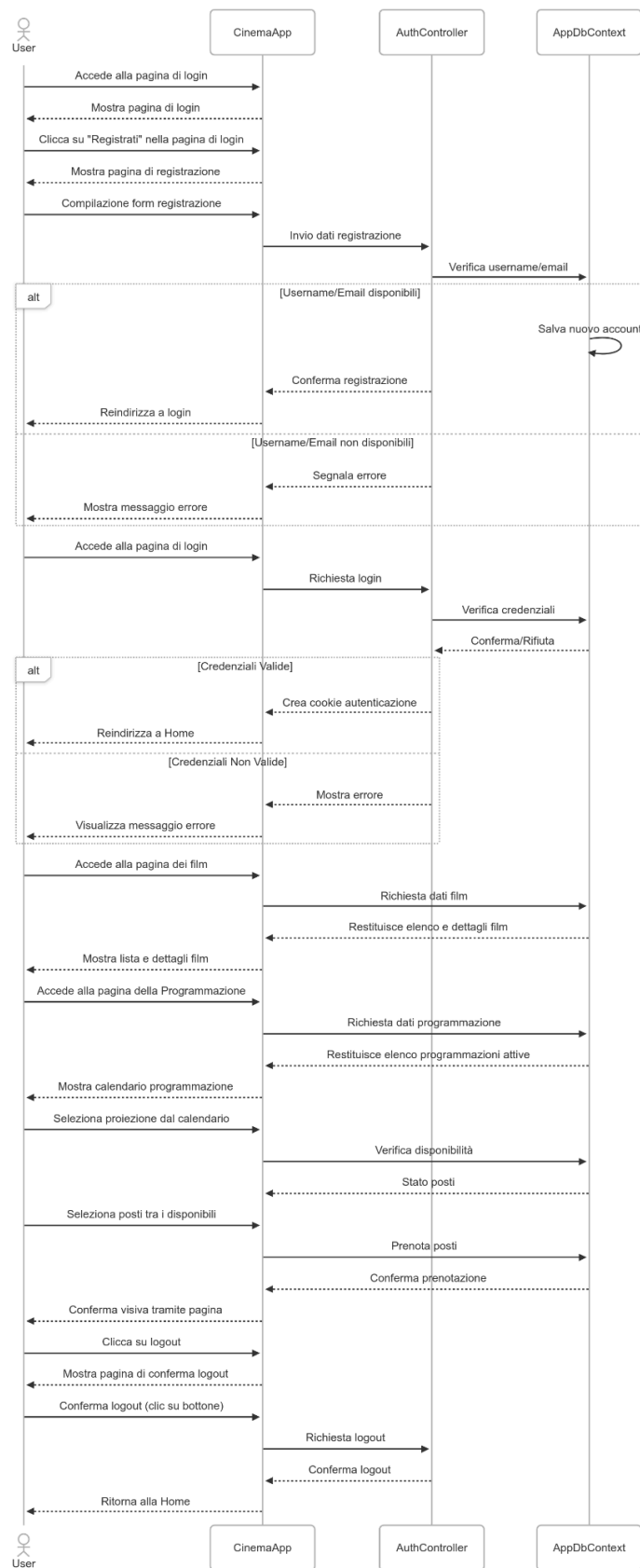
- Posti: elenca i posti a sedere in ciascuna sala, indicando riga e numero e collegandoli alla sala corrispondente.
 - Proiezioni: associa un film a una sala e a un orario specifico, con informazioni sul prezzo base e un flag che indica se la proiezione è attiva o meno.
 - Biglietti: memorizza le prenotazioni, con dati come la proiezione scelta, il posto assegnato, il prezzo finale, il codice di prenotazione, la data di acquisto, l'email del cliente e un flag per verificare se il biglietto è stato utilizzato.
-

Migliorie e funzionalità future

Per rendere il sistema ancora più completo e funzionale, alcune delle possibili evoluzioni includono:

- L'implementazione di tabelle con funzionalità avanzate, come ordinamento personalizzato, ricerca per nome, limitazione delle righe visualizzate e paginazione integrata.
- Sostituire SQLite con PostgreSQL o SQL Server permetterebbe di gestire transazioni concorrenti e grandi volumi di dati, sfruttando funzionalità avanzate, per garantire prestazioni e affidabilità superiori.
- La creazione di report dettagliati sulle vendite, arricchiti da grafici e statistiche per monitorare le performance.
- L'integrazione di codici a barre all'interno dei biglietti PDF per migliorare il controllo degli accessi.
- Lo sviluppo di un servizio automatico per l'invio via email dei biglietti acquistati.
- L'aggiunta di una sezione dedicata alla gestione delle iscrizioni alla newsletter e un modulo per l'invio e il tracciamento dei reclami.
- Un sistema interno di valutazione dei film, accessibile agli utenti solo dopo aver utilizzato il biglietto per vedere la proiezione, permettendo così recensioni più attendibili.
- Infine, il miglioramento del design per renderlo completamente responsive e ottimizzato per dispositivi mobile, così da offrire un'esperienza fluida e piacevole su qualsiasi schermo.

FlowChart User



FlowChart Admin

